

Report

Of

CYBER SECURITY INTERNSHIP

(Codec Technologies Pvt. Ltd. – AICTE & ICAC Approved Internship Programme)

Project Work: Network Traffic Analysis and Cryptographic Implementation

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Roll Number of Student: 25SCS1003005170

Name of Student: Yuvraj

Class / Section: 2CSE20

Batch: 2025 – 2029

Academic Year 2026–27

CANDIDATE'S DECLARATION

I, Yuvraj, a student of B.Tech Computer Science and Engineering (Section 2CSE20, Roll No. 25SCS1003005170) at IILM University, Greater Noida, do hereby declare that this report, titled “Cyber Security Internship – Network Traffic Analysis and Cryptographic Implementation,” has been prepared by me on the basis of the one-month Cyber Security internship programme completed with Codec Technologies Pvt. Ltd., an AICTE and ICAC approved internship programme conducted from 08 June 2026 to 08 July 2026.

I further declare that all work described herein, including the design, implementation, and testing of the network traffic analyser and the cryptography and hashing engine, was carried out independently by me using the Python programming language and its standard library. The packet analysis module operates exclusively on a self-contained, hard-coded sample packet constructed for demonstration purposes; at no point was live network traffic captured, intercepted, or analysed on any network belonging to a third party, and no system was accessed without authorisation.

All references used in the preparation of this report have been duly acknowledged, and I have not copied material from any other source without appropriate citation. I affirm that this report is my own work and has not been submitted, in part or in full, to any other institute for the award of any other degree or diploma. I shall be solely responsible for any copyright violation in this regard, if found.

Signature

Student Name : Yuvraj

Roll No. : 25SCS1003005170

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the completion of my Cyber Security internship with Codec Technologies Pvt. Ltd. This one-month programme gave me an opportunity to gain practical exposure to the fundamentals of network protocol analysis and applied cryptography, and to translate classroom concepts into working software.

I am thankful to Codec Technologies Pvt. Ltd. for structuring this AICTE and ICAC approved internship around clear, task-based objectives that allowed me to work through realistic security problems in a self-directed manner. I would particularly like to thank Dr. Anurag Shrivastava, Program Manager, for the guidance and coordination provided during the internship period.

During the internship, I worked on two connected deliverables: a network traffic analyser and packet inspector capable of decoding Ethernet and IPv4 headers and flagging unencrypted application-layer traffic, and a cryptography and hashing engine demonstrating SHA-256 integrity verification, symmetric encryption, and the mathematical basis of asymmetric key generation. Working on these tasks helped me understand how data is structured on the wire and how cryptographic primitives protect that data in transit and at rest.

I would also like to express my sincere gratitude to my teachers and academic mentors at IILM University, Greater Noida, for their continuous support and encouragement, and for providing an academic environment that allowed me to pursue this internship programme alongside my regular coursework. Their guidance helped me maintain a systematic and disciplined approach towards completing this project work.

Finally, I express my heartfelt gratitude to my parents and family for their constant encouragement, support, and blessings, which helped me stay focused and complete this project work successfully.

Signature

Student Name : Yuvraj

Roll No. : 25SCS1003005170

CERTIFICATE / COMPLETION EVIDENCE & Offer Letter

This section contains the official completion evidence relating to the Cyber Security internship programme described in this report.

The Certificate of Internship reproduced below was issued to the candidate by Codec Technologies Pvt. Ltd. in recognition of the successful completion of the one-month AICTE and ICAC approved internship programme in the role of Cyber Security Intern, conducted from 08 June 2026 to 08 July 2026. The certificate is signed by Dr. Anurag Shrivastava, Program Manager, and carries the issuing organisation's AICTE Corporate ID.



INTERNSHIP OFFER LETTER

Dear Yuvraj,

We are pleased to offer you a 1 Month Internship as Project Intern at Codec Technologies, a global platform dedicated to empowering learners and connecting diverse talent to create meaningful career opportunities worldwide. Codec Technologies delivers dedicated IT and business consultancy services across 27+ countries, empowering global innovation and strategic growth.

Internship Details:

- Designation : Cyber Security Intern
- Location & Mode: Pan India, Hybrid
- Duration : 08/06/2026 to 08/07/2026
- Reporting to : Assigned Project Head(s)

This program provides an immersive experience, enabling you to gain industry-level knowledge and acquire valuable skills. Exceptional interns may also qualify for a scholarship award based on weekly performance reviews conducted by project supervisor.

If you accept this offer and the terms above, please complete your assigned projects and training tasks on our platform and via email.

Best regards,

Sincerely,

Dr. Anurag Shrivastava,
Codec Technologies India
NCS ID: EI9E86-0116588288923
Date Of Issue - 08/06/2026



Dr. Anurag Shrivastava
Talent Acquisition Manager

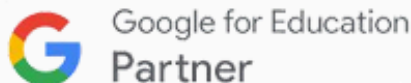


TABLE OF CONTENTS

1. Candidate's Declaration
2. Acknowledgement
3. Certificate / Completion Evidence
4. Table of Contents
5. Project Description
 - 5.1 Introduction
 - 5.2 About the Internship Programme / Codec Technologies
 - 5.3 Problem Statement
 - 5.4 Project Objectives
 - 5.5 Scope of the Work
 - 5.6 Technologies and Tools Used
 - 5.7 System Architecture
 - 5.8 Methodology
 - 5.9 Learning Outcomes
 - 5.10 Documentation / Completion Evidence
6. References

5. PROJECT DESCRIPTION

This report documents the work completed as part of a one-month Cyber Security internship with Codec Technologies Pvt. Ltd., an AICTE and ICAC approved internship programme. The internship consisted of two connected deliverables: a Network Traffic Analyzer and Packet Inspector, which decodes raw frame data and flags insecure application-layer traffic, and a Cryptography and Hashing Implementation, which demonstrates the three fundamental categories of cryptographic operation — hashing, symmetric encryption, and asymmetric key generation.

The work progressed from the study of network protocol structure and cryptographic theory to the design, implementation, and testing of two self-contained Python programs. Both deliverables were developed independently by the candidate using only the Python standard library, without external packages, live network capture, or third-party services. Together they illustrate a common progression in security engineering: first understanding how data travels and where it is exposed, and then applying cryptographic controls to protect it.

This report is organized into subsections covering the introduction, the nature of the internship programme, the problem statement, project objectives, scope, technologies used, system architecture, methodology, learning outcomes, and supporting documentation.

5.1 Introduction

Information security rests on two closely related foundations: an accurate understanding of how data is structured and transmitted across a network, and the correct application of cryptographic techniques to preserve the confidentiality and integrity of that data. A security professional who cannot read a packet header cannot reason about what an attacker sees on the wire; equally, a developer who cannot distinguish a hash from a cipher, or a symmetric scheme from an asymmetric one, cannot select an appropriate control for a given risk.

The Cyber Security internship provided an opportunity to engage with both foundations in a structured, deliverable-based format. Rather than focusing purely on theoretical concepts, the internship required the completion of two practical programs that, together, illustrate the defensive lifecycle: observing traffic to detect an exposure, and then applying cryptography to remove it.

The first deliverable involved building a packet inspector in Python that unpacks a raw Ethernet frame into its constituent fields, validates the EtherType, decodes the IPv4 header to recover the time-to-live value, the transport protocol identifier, and the source and destination addresses, and finally inspects the remaining payload for evidence of unencrypted application-layer traffic.

The second deliverable involved building a cryptography engine exposing three capabilities: SHA-256 digest generation for integrity verification, a repeating-key XOR stream cipher for symmetric encryption and decryption, and a textbook RSA key-pair simulation illustrating how public and private exponents are derived from a pair of prime numbers.

Together, these two deliverables provided practical exposure to binary data handling, protocol decoding, cryptographic hashing, symmetric and asymmetric cryptography, and the responsible, laboratory-scoped construction of security tooling. The following subsections describe the programme, the problem addressed, the objectives pursued, and the methodology followed in greater detail.

5.2 About the Internship Programme / Codec Technologies

Codec Technologies Pvt. Ltd. offers structured, industry-oriented internship programmes intended to give students practical exposure to applied technology domains. The Cyber Security internship undertaken by the candidate is an AICTE and ICAC approved programme of one month's duration, conducted from 08 June 2026 to 08 July 2026, in the role of Cyber Security Intern.

The programme is organised around defined technical deliverables rather than passive instruction. Participants are given problem briefs covering areas such as network fundamentals, protocol analysis, and applied cryptography, and are expected to produce working code, test it, and document their findings. Progress is coordinated by a programme manager, and completion is recognised through an issued certificate of internship.

The domain of the internship — cyber security — carries particular responsibilities. Tools that inspect network traffic or manipulate cryptographic material can be misused, and the internship work was therefore deliberately scoped so that it could not be. The packet analyser operates on a hard-coded, synthetic packet constructed by the candidate, not on captured traffic; it does not open a raw socket, does not require elevated privileges, and does not touch any network interface. The cryptography engine encrypts only sample text supplied within the script itself. Both programs are educational artefacts intended to demonstrate understanding of the underlying mechanisms.

The programme is valuable from an academic perspective because it exposes participants to a real engineering constraint that classroom exercises often omit: security concepts must be implemented correctly, tested honestly, and documented with their limitations clearly stated.

5.3 Problem Statement

Data transmitted across a network is encapsulated in a series of nested protocol headers. A single frame carries link-layer addressing, network-layer addressing and routing metadata, transport-layer port information, and finally the application payload. To a program, this arrives as an undifferentiated sequence of bytes. Extracting meaning from it requires knowledge of exactly how many bytes each field occupies, in what order, and in which byte ordering convention — information that is specified in protocol standards but not evident from the data itself.

A first problem, therefore, was to construct a program capable of decomposing a raw frame into human-readable fields, correctly handling fixed-width fields such as the six-byte MAC addresses, the network byte order of the EtherType value, and the variable-length IPv4 header whose true length must be derived from the low four bits of the first octet rather than assumed.

A second problem concerns exposure. Once the payload has been isolated, the question arises of whether its contents are protected. Application protocols that transmit in cleartext — most notably plain HTTP — expose credentials, session identifiers, and request paths to anyone positioned on the path. A security tool should be able to recognise this condition and raise an alert, without needing to decrypt anything.

A third, related problem concerns the remedy. Having identified that data is exposed, a practitioner must be able to apply the appropriate cryptographic control, and must understand which control addresses which property. Hashing establishes integrity but not confidentiality and is not reversible; symmetric encryption provides confidentiality but requires both parties to already share a secret; asymmetric encryption solves the key distribution problem at a much higher computational cost.

The overall problem addressed in this project was therefore twofold: first, to decode raw network data reliably and detect an unencrypted application-layer exposure within it; and second, to implement and demonstrate the three principal categories of cryptographic operation that address such exposures, at a level where the internal mechanics of each remain visible and traceable.

5.4 Project Objectives

The objectives of the internship project work were defined around the two deliverables undertaken. The major objectives were as follows:

1. To understand the layered encapsulation model by which application data is wrapped in transport, network, and link-layer headers before transmission.
2. To study the structure of the Ethernet II frame header and to implement its decomposition into destination MAC address, source MAC address, and EtherType fields using Python's struct module.
3. To convert raw six-byte hardware addresses into the conventional colon-separated hexadecimal notation used by network administrators.
4. To handle network byte order correctly when interpreting multi-byte protocol fields, using socket.htons for the EtherType value.
5. To study the IPv4 header format and to implement extraction of the version and Internet Header Length nibbles, the time-to-live value, the transport protocol identifier, and the source and destination IP addresses.
6. To compute the true IPv4 header length from the IHL field so that the payload boundary is derived from the packet rather than assumed, correctly handling headers that carry options.
7. To implement a payload inspection routine that decodes the application-layer bytes defensively and identifies indicators of unencrypted HTTP traffic.
8. To produce a clear, formatted console report summarising the decoded frame and the outcome of the security analysis.
9. To implement SHA-256 digest generation using Python's hashlib module and to explain its role in verifying data integrity.
10. To implement a symmetric repeating-key XOR cipher, together with its inverse, and to represent ciphertext in hexadecimal form for safe display and storage.
11. To implement a textbook RSA key-pair simulation demonstrating how a modulus and a pair of public and private exponents are derived from two prime numbers.
12. To test both programs, observe their output, and document their behaviour, assumptions, and limitations honestly, including the known cryptographic

weaknesses of the demonstration cipher.

5.5 Scope of the Work

The scope of this project is limited to two connected deliverables undertaken during the Cyber Security internship. The first, the network traffic analyser, and the second, the cryptography and hashing implementation, together define the boundaries of the work described in this report.

Scope of the Network Traffic Analyzer

The analyser decodes Ethernet II frames carrying IPv4 packets. It reports the source and destination MAC addresses, the EtherType, the source and destination IP addresses, and the transport protocol identifier, and it performs a keyword-based inspection of the application payload to detect cleartext HTTP. It operates on a single hard-coded sample frame defined within the script.

The scope does not extend to live packet capture, raw socket binding, promiscuous-mode interface access, IPv6, ARP, VLAN-tagged frames, TCP or UDP header parsing, session reassembly, or any form of traffic modification, injection, or interception. The tool is a passive decoder of a fixed sample and has no network capability whatsoever.

Scope of the Cryptography and Hashing Implementation

The cryptography engine covers three operations: SHA-256 hashing of a text string, symmetric encryption and decryption of a text string using a repeating-key XOR cipher with hexadecimal output, and the generation of an RSA-style key pair from a fixed pair of small primes.

The scope does not extend to production-grade ciphers such as AES, authenticated encryption modes, secure key derivation, secure key storage, random number generation for real keys, digital signatures, certificate handling, or cryptographically meaningful RSA parameters. The XOR cipher is included specifically to make the mechanics of symmetric encryption visible and is expressly not proposed as a secure scheme; the RSA component simulates key derivation only and does not perform encryption or decryption.

Overall, the scope of this project is intentionally limited to demonstrate the fundamentals of protocol decoding and cryptographic operation, rather than to deliver a production-ready security monitoring or encryption system.

5.6 Technologies and Tools Used

The completion of the internship deliverables involved a deliberately minimal technology stack, selected so that every operation performed by the code would remain visible rather than being delegated to a library abstraction.

5.6.1 Programming Language

- Python 3 — used as the sole implementation language for both deliverables, chosen for its readable syntax and its comprehensive standard library, which made external

dependencies unnecessary.

5.6.2 Network Analysis Modules

- `struct` — used to unpack fixed-width binary fields from the raw frame according to an explicit format specification, covering the two six-byte hardware addresses and the two-byte `EtherType` of the Ethernet header, and the time-to-live, protocol, and address fields of the IPv4 header.
- `socket` — used for two protocol-aware conversions: `socket.htons` to interpret the `EtherType` in the correct byte order, and `socket.inet_ntoa` to render four-byte binary addresses in dotted-decimal notation.
- Python byte slicing and bitwise operators — used to isolate the Internet Header Length nibble from the first octet of the IPv4 header and to compute the payload offset.

5.6.3 Cryptography Modules

- `hashlib` — used to generate SHA-256 digests, providing a standards-compliant implementation of the hash function for the integrity demonstration.
- `bytearray` and the XOR operator — used to implement the symmetric stream cipher byte by byte, with modular indexing into the key so that the key repeats across the length of the message.
- Python's hexadecimal conversion facilities — `bytearray.hex()` and `bytes.fromhex()` — used to render ciphertext in a printable form and to recover the raw bytes for decryption.

5.6.4 Development Environment

- A standard Python 3 interpreter and text editor were used for development; both programs are executed from the command line and print their results to standard output.

No third-party packages, packet capture libraries, cryptographic frameworks, machine learning libraries, or external web APIs were used in either deliverable. This reflects the intentionally transparent, fundamentals-first scope defined for the internship.

5.7 System Architecture

Although the two deliverables serve different purposes, both follow a structured flow in which raw input is processed in defined stages to produce a meaningful output. The architecture of each is described separately below.

5.7.1 Network Traffic Analyzer Architecture

The analyser follows a linear decoding pipeline, beginning with an undifferentiated sequence of bytes and ending with a security verdict:

Raw Frame Bytes → Ethernet Header Parsing → `EtherType` Validation →

IPv4 Header Parsing → Payload Extraction → Security Heuristic → Console Report

At each stage the data becomes progressively more interpretable. The first fourteen bytes are unpacked into the two hardware addresses and the EtherType, and the remainder is passed forward as the network-layer payload. The EtherType is then checked to confirm that the frame carries an IPv4 packet before any attempt is made to decode it as one; a frame of any other type is not misinterpreted.

The IPv4 stage reads the first octet to recover the Internet Header Length, multiplies it by four to obtain the header size in bytes, and unpacks the time-to-live, protocol identifier, and the two four-byte addresses from their fixed offsets. Because the payload boundary is computed from the header length rather than assumed to be twenty bytes, the parser remains correct for packets carrying IP options.

The final stage applies a simple heuristic to the application payload. The bytes are decoded as UTF-8 with invalid sequences ignored, so that binary or encrypted content cannot raise an exception, and the resulting text is searched for characteristic HTTP tokens. A match produces an explicit warning that unencrypted traffic has been detected; the absence of a match produces a neutral statement that the payload is encrypted or in a binary format, which is the honest conclusion, since the tool cannot distinguish between those two cases.

5.7.2 Cryptography Engine Architecture

The cryptography engine is organised as a single class exposing three independent static operations, each processing its input through a defined sequence:

Input Text → UTF-8 Encoding → SHA-256 Compression → Hexadecimal Digest

Plaintext + Key → Repeating-Key XOR → Hex Ciphertext → XOR Inverse → Plaintext

Prime Pair (p, q) → Modulus n → Public Exponent e → Private Exponent d → Key Pair

The hashing path is one-directional by design: the input is encoded to bytes, compressed to a fixed 256-bit digest, and rendered as a sixty-four character hexadecimal string. Any change to the input, however small, produces an entirely different digest, which is the property that makes the operation useful for integrity verification.

The symmetric path is reversible. Each plaintext byte is combined with the corresponding key byte using the exclusive-or operation, with the key index taken modulo the key length so that a short key extends across a longer message. Because exclusive-or is its own inverse, the same routine applied to the ciphertext with the same key recovers the original text — a property the program demonstrates directly by decrypting its own output and displaying the result.

The asymmetric path produces a key pair rather than performing encryption. Two primes are multiplied to form the modulus, a public exponent is selected, and the corresponding private exponent is presented as its modular inverse with respect to the totient. The two exponents share the modulus but are otherwise independent, which is the structural property that allows one to be published while the other remains secret.

This architecture keeps each cryptographic operation self-contained and easy to trace, which was appropriate given the educational scope defined for the internship.

5.8 Methodology

The project was completed through a structured, stage-by-stage methodology that progressed from requirement understanding to final documentation. The methodology followed for both deliverables is described below.

5.8.1 Requirement Understanding

The first stage involved reviewing the internship briefs to understand the required capabilities of each deliverable, the constraint that only the Python standard library be used, and the requirement that no live or third-party network be involved in any part of the work.

5.8.2 Protocol Study

The structure of the Ethernet II frame and the IPv4 header was studied field by field, recording the byte offset, width, and byte-ordering convention of each field required by the analyser. This produced the format specifications later supplied to `struct.unpack`.

5.8.3 Construction of a Sample Frame

Rather than capturing traffic, a synthetic frame was constructed by hand as a byte literal within the script. It comprises a fourteen-byte Ethernet header with an IPv4 EtherType, an IPv4 header carrying a TCP protocol identifier and a private source and destination address pair, and an application payload consisting of a plain HTTP request line and host header. This approach made the expected output known in advance, which in turn made incorrect parsing immediately visible.

5.8.4 Implementation of the Analyzer

The decoding functions were implemented in sequence — frame parsing, address formatting, IPv4 parsing, and payload inspection — each returning the remaining unparsed bytes so that the next stage could continue from the correct offset. A reporting routine was then added to present the decoded fields in a readable, labelled format.

5.8.5 Cryptographic Study and Implementation

The three cryptographic categories were studied in terms of the security property each provides, and were then implemented in turn: hashing using `hashlib`, symmetric encryption as a repeating-key XOR routine with a matching inverse, and asymmetric key derivation as a fixed-parameter RSA simulation.

5.8.6 Testing and Verification

Each program was executed and its output compared against expectation. For the analyser, the decoded addresses and protocol identifier were checked against the values deliberately placed in the sample frame, and the payload heuristic was confirmed to raise the expected warning. For the cryptography engine, the SHA-256 digest was verified to be stable across runs and to change completely on any modification of the input, and the

symmetric cipher was verified by a round-trip test in which the decrypted output was compared against the original plaintext.

5.8.7 Review of Limitations

A deliberate review stage was carried out to identify and record where each program falls short of production quality, so that the limitations would be stated in the documentation rather than discovered by a reader. The principal findings of this review are recorded in the subsection below.

5.8.8 Documentation

Finally, the design, functionality, test results, and known limitations of both deliverables were documented in a structured, readable format, culminating in this report.

5.9 Learning Outcomes

Completion of the Cyber Security internship contributed to the development of the following practical skills and areas of understanding:

- Practical understanding of protocol encapsulation and of how application data is nested within transport, network, and link-layer headers.
- Ability to decode binary protocol data in Python using struct format specifications, byte slicing, and bitwise operations.
- Understanding of network byte order and of why multi-byte protocol fields must be converted before interpretation.
- Understanding of the IPv4 header layout, including why the header length must be read from the IHL field rather than assumed to be a fixed twenty bytes.
- Awareness of the security exposure created by cleartext application protocols and of how such exposure is visible to any party on the network path.
- Practical experience with defensive decoding — handling arbitrary binary input without allowing malformed data to crash the program.
- Understanding of the distinction between hashing, symmetric encryption, and asymmetric encryption, and of the specific security property each provides.
- Practical experience implementing a symmetric cipher and its inverse, and verifying correctness through round-trip testing.
- Understanding of the mathematical structure underlying RSA key generation and of why the public and private exponents can share a modulus safely.
- A clear appreciation of the limitations of the implemented schemes — in particular that a repeating-key XOR cipher is vulnerable to key reuse and known-plaintext analysis, and that RSA parameters of the size used here offer no real security — and of the importance of stating such limitations explicitly rather than overstating what a prototype achieves.
- Awareness of the ethical and legal boundaries surrounding security tooling, and of the importance of scoping such tools to authorised, self-contained test data.
- Improved technical documentation and communication skills through the preparation of structured project documentation.

5.10 Documentation / Completion Evidence

The Cyber Security internship was completed with Codec Technologies Pvt. Ltd. over a period of one month, from 08 June 2026 to 08 July 2026. The work completed comprises two deliverables: the network traffic analyser and packet inspector, and the cryptography and hashing implementation.

The following project files and records serve as supporting evidence of the completed work:

- network_analyzer.py — the complete packet inspection program, containing the Ethernet frame parser, the IPv4 header parser, the payload security heuristic, and the reporting routine.
- crypto_implementation.py — the complete cryptography engine, containing the SHA-256 hashing method, the symmetric XOR cipher and its inverse, the RSA key simulation, and the demonstration routine.
- Recorded console output of both programs, showing the decoded frame fields, the unencrypted-traffic warning, the generated digest, the ciphertext and recovered plaintext, and the generated key pair.
- Certificate of Internship issued by Codec Technologies Pvt. Ltd., reproduced in the Certificate / Completion Evidence section of this report.

As shown in that section, the formal certificate for this internship has been issued and is included in full, together with the issuing organisation's AICTE Corporate identification and the signature of the Program Manager.

6. REFERENCES

1. Codec Technologies Pvt. Ltd., Cyber Security Internship Programme — task briefs, deliverable requirements, and reference materials provided during the internship conducted from 08 June 2026 to 08 July 2026.
2. Internet Engineering Task Force, RFC 791 — Internet Protocol, DARPA Internet Program Protocol Specification, reference for the IPv4 header format, the Internet Header Length field, the time-to-live field, and the protocol identifier, [ietf.org](https://www.ietf.org/rfc/rfc791.txt).
3. Internet Engineering Task Force, RFC 894 — A Standard for the Transmission of IP Datagrams over Ethernet Networks, reference for Ethernet frame structure and EtherType values, [ietf.org](https://www.ietf.org/rfc/rfc894.txt).
4. IEEE 802.3 Standard for Ethernet — reference for the Ethernet II frame header and the 48-bit MAC address format, [ieee.org](https://www.ieee.org/).
5. Internet Engineering Task Force, RFC 9110 and RFC 9112 — HTTP Semantics and HTTP/1.1, reference for the request-line and header syntax used in the payload inspection heuristic, [ietf.org](https://www.ietf.org/rfc/rfc9110.txt).
6. National Institute of Standards and Technology, FIPS PUB 180-4 — Secure Hash Standard, reference for the SHA-256 algorithm used in the integrity demonstration, nist.gov.
7. R. L. Rivest, A. Shamir and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” Communications of the ACM, reference for the RSA key generation scheme simulated in this project.
8. Python Software Foundation, Python Documentation — reference material for the struct, socket, and hashlib modules and for byte and bytearray handling, docs.python.org.
9. W. Stallings, Cryptography and Network Security: Principles and Practice — general reference for the classification of cryptographic primitives and the properties of symmetric and asymmetric schemes.
10. A. S. Tanenbaum and D. J. Wetherall, Computer Networks — general reference for protocol layering and encapsulation.